Biased Randomisation in a Savings-Based GRASP: Four Selection Rules for the Travelling Salesman Problem

Heuristic Algorithms in Transport and Finance — Master's course assignment
Javier Salvatierra
Jesús Inurria

October 2026

Abstract

A GRASP repeats a randomised greedy construction and a local search. We build travelling salesman tours with the Clarke and Wright savings, improve them with 2-opt and compare the classic uniform restricted candidate list (RCL) with four *biased randomisation* rules — geometric, triangular, Zipf and softmax — whose probability decreases along the savings list. Each rule is tuned on three separate instances and tested on seven TSPLIB instances with 51–280 cities, 20 runs of 150 iterations each. The classic GRASP lowers the mean gap of the greedy savings tour plus 2-opt from 5.07 % to 2.22 %; the geometric rule lowers it to 1.70 % at the same cost per run, and is significantly better on two instances and never significantly worse. A Friedman test across instances detects no global difference ($p = 0.76$).

1 Introduction

A salesman has to visit n cities once each and return home along the shortest possible tour. This is the Travelling Salesman Problem (TSP), the standard NP-hard routing problem: exact methods need specialised codes whose running time grows quickly with n, so heuristics are used when a good tour is needed fast. A Greedy Randomized Adaptive Search Procedure (GRASP) [2] is a multi-start method: every iteration builds a solution by choosing, at each step, one of the best candidates at random, and improves it by local search; the best solution found is returned.

The classic GRASP draws uniformly from a *restricted candidate list* (RCL). Juan et al. [3] used instead, in the savings heuristic of Clarke and Wright [1], a geometric distribution over the positions of the sorted savings list, so that better candidates are always more likely; this is *biased randomisation*. We compare four biased rules with the classic RCL in a GRASP for the TSP, with everything else fixed. We deal only with the symmetric Euclidean TSP and a plain 2-opt local search.

We test on seven TSPLIB instances [4] whose optimal tours are published: `eil51`, `berlin52`, `st70`, `eil76`, `kroA100`, `ch130` and `a280`. The number in the name is the number of cities, so `a280` has $n = 280$. All are of type `EUC_2D` (city coordinates), and the optimum is the length of the companion tour `.opt.tour`. Below we state the model and the method, and report quality, time and significance tests.

2 The TSP and the Clarke and Wright savings

Let $V = \{1, \dots, n\}$ be the cities and $d(i, j) = d(j, i)$ the Euclidean distance between i and j, rounded to the nearest integer as TSPLIB prescribes for its published optima. A tour is a permutation π of V, and the TSP reads

$$\min_{\pi} \sum_{t=1}^{n} d(\pi_t, \pi_{t+1}) \qquad \text{s.t.} \qquad \pi = (\pi_1, \dots, \pi_n) \text{ is a permutation of } V, \qquad \pi_{n+1} = \pi_1, \tag{1}$$

i.e. every city is visited exactly once and the tour closes at its starting city.

Savings were designed for vehicle routing, with a depot. For the TSP we choose a *hub* h, the city closest to the centre of gravity of the coordinates, and start from the *dummy solution* that goes from h to every other city and straight back. Serving two cities i and j in sequence instead avoids one return to the hub and one departure from it, and saves

$$s(i, j) = d(h, i) + d(h, j) - d(i, j). \tag{2}$$

By the triangle inequality $s(i, j) \geq 0$ up to rounding; savings are largest for cities that are close to each other and far from the hub. A saving depends only on h, i and j, never on the partial solution: as long as i and j are ends of different paths, both are still joined to the hub, and linking them saves exactly $s(i, j)$. The $\binom{n-1}{2}$ savings are therefore computed and sorted in decreasing order once per instance (4851 pairs for `kroA100`); this *savings list* is the candidate list of every GRASP iteration.

3 GRASP with savings: RCL versus biased randomisation

The GRASP construction starts with every city except the hub as a path of its own and repeats three steps:

1. **Filter.** Keep only the pairs (i, j) whose two cities are ends of their paths and lie on different paths. Linking an interior city would create a branch; linking two ends of the same path would close a cycle that skips the hub. A pair that fails can be deleted for good: paths only grow, so an interior city stays interior and two cities on the same path stay together.
2. **Choose** a position k in the filtered list (0 for the largest saving) with the selection rule.
3. **Merge** the two paths through the chosen pair.

Table 1: Mean gap (%) to the TSPLIB optimum over 20 runs of 150 iterations. *CW*: greedy savings tour ($\alpha = 0$), a single run, without and with 2-opt. Superscripts: $^+$ significantly better, $^-$ significantly worse than the RCL (Wilcoxon rank-sum, Holm, 5 %). Better/equal/worse counts these outcomes (equal: no significant difference). Last row: mean wall-clock time per run on `a280`.

Instance	n	Optimum	CW	CW + 2-opt	RCL	Geometric	Triangular	Zipf	Softmax
`eil51`	51	426	5.40	4.46	1.41	1.10	0.95^+	1.20	1.41
`berlin52`	52	7 542	11.06	6.58	3.69	0.93^+	2.04^+	3.35	3.44
`st70`	70	675	4.74	3.56	0.82	1.01	0.99	1.26^-	0.93
`eil76`	76	538	5.02	3.53	1.78	1.65	1.66	2.02	1.89
`kroA100`	100	21 282	6.04	3.18	0.94	1.14	1.12	1.19	0.91
`ch130`	130	6 110	7.28	4.70	1.71	1.73	1.80	1.54	1.46
`a280`	280	2 579	13.65	9.46	5.16	4.31^+	6.14^-	4.74	5.70
Mean			**7.60**	**5.07**	**2.22**	**1.70**	**2.10**	**2.18**	**2.25**
Better/equal/worse vs. RCL					—	2/5/0	2/4/1	0/6/1	0/7/0
Time per run, `a280` (s)					5.81	5.33	5.53	10.18	9.93

After $n-2$ merges a single path covers the $n-1$ cities, and joining its two ends to the hub closes the tour. The list shrinks after every merge, the only *adaptive* element here; always taking the largest saving gives the classic savings heuristic. Let m be the length of the filtered list and s_k its savings (2). The five rules are:

- **RCL (classic GRASP).** Uniform choice among the candidates with $s_k \geq s_0 - \alpha\,(s_0 - s_{m-1})$: a hard cut-off on the *value* of the saving.
- **Geometric.** $P(k) \propto \beta(1-\beta)^k$, truncated to the m positions; with $\beta = 0.4$ the best pair is chosen with probability 0.4, the second with 0.24.
- **Triangular.** A density that decreases linearly over the first $L = \lceil fm \rceil$ positions (at least 3) and is zero afterwards.
- **Zipf.** $P(k) \propto (k+1)^{-e}$, a power law with a heavier tail than the geometric one.
- **Softmax.** $P(k) \propto \exp(z_k/T)$ with $z_k = (s_k - s_0)/(s_0 - s_{m-1}) \in [-1, 0]$: it uses the normalised *value* of the saving, not its position.

Geometric, Zipf and softmax have no cut-off: every feasible pair keeps some probability; the triangular rule has one, but on the rank, not the value. Geometric, triangular and Zipf look only at the rank, so their diversity does not depend on how the saving values are distributed; the RCL and softmax do. Each rule has one parameter to tune, from greedy (every iteration builds the same tour) to nearly uniform (poor tours).

The local search is *2-opt*: for two non-adjacent edges (a,b) and (c,d) of the tour, if $d(a,c)+d(b,d) < d(a,b)+d(c,d)$ the segment $b \ldots c$ is reversed, so that (a,c) and (b,d) replace them. The first improving move is applied until none is left.

4 Implementation

The code is a single notebook, `GRASP_TSP_savings_biased_EN.ipynb`, in Python 3.13 with NumPy, SciPy and pandas. The savings list is three arrays sorted with a stable sort. The construction keeps the degree and the path index of every city, so the filter of step 1 is one vectorised mask over the remaining list. The geometric and triangular rules sample the position directly by inverting their distribution functions; Zipf and softmax build a weight for every candidate and sample from the cumulative sum. The 2-opt, sequential by nature, is compiled with `numba`.

Tuning. Each rule has a grid of 6–8 values from almost greedy to very random, and every grid value receives the same effort: three instances *not* used for testing (`kroB100`, `eil101`, `ch150`; the optimum of `kroB100`, 22 141, is fixed in the code), with 6 runs of 80 iterations and seeds different from the test ones. The value with the lowest mean gap is kept: $\alpha = 0.05$, $\beta = 0.4$, $f = 0.002$, $e = 3$ and $T = 0.02$, none at the edge of its grid. Tuning took 306 s. Juan et al. [3] draw β from $U(0.05, 0.25)$ at every selection, need no tuning and apply no 2-opt; with 2-opt after every construction, our tuning prefers a fixed and greedier $\beta = 0.4$ (0.2 was also in the grid).

Protocol. Each rule runs 20 times on each test instance, 150 iterations per run, with seeds $0, \ldots, 19$ shared by all rules, the same hub and the same 2-opt: 700 runs in 1011 s. We record the final cost (the objective of (1)), gap $= 100\,(\text{cost} - \text{opt})/\text{opt}$, and the wall-clock time of the run (150 constructions and local searches). Each biased rule is compared with the RCL instance by instance with the two-sided Wilcoxon rank-sum test on the 20 final costs, with Holm correction over the 28 comparisons at the 5 % level; the Friedman test compares the five rules at once on the mean gaps, with the instances as blocks. As a reference we also run the RCL with $\alpha = 0$ (ties broken at random with a fixed seed), with and without 2-opt, and check that each constructed tour is a permutation of the cities.

5 Computational results

Table 1 gives the mean gap of the 20 runs for every rule and instance, and Figure 1 their distribution.

GRASP works. The greedy savings tour ($\alpha = 0$) is 7.60 % above the optimum on average, and 2-opt brings it to 5.07 %. With a randomised construction and 150 iterations the mean gap falls to 2.10–2.25 % with any rule but the geometric one, and to 1.70 % with it; the classic GRASP removes 56 % of the remaining error. The mechanism is diversity: each randomised tour leads 2-opt to a different local optimum, and only the best of 150 is kept.

Geometric vs. RCL. The geometric rule has the lowest mean gap, 23 % below the RCL, and the lowest best-of-20 gap averaged over the instances (0.81 % against 1.70 % for the RCL). It is significantly better on `berlin52` (3.69 → 0.93 %) and `a280` (5.16 → 4.31 %), not significantly different on the other five; a run of the geometric rule beats a run of the RCL (ties counted as one half) with mean probability 0.60. A likely reason is that the RCL cut-off is set on the normalised saving value, so with a single α the number of candidates it admits depends on how the values are spread at the head of the list: on `eil51` the mean and the best gap of the RCL are both 1.41 %, so the 20 runs end with the same cost, 432 (the

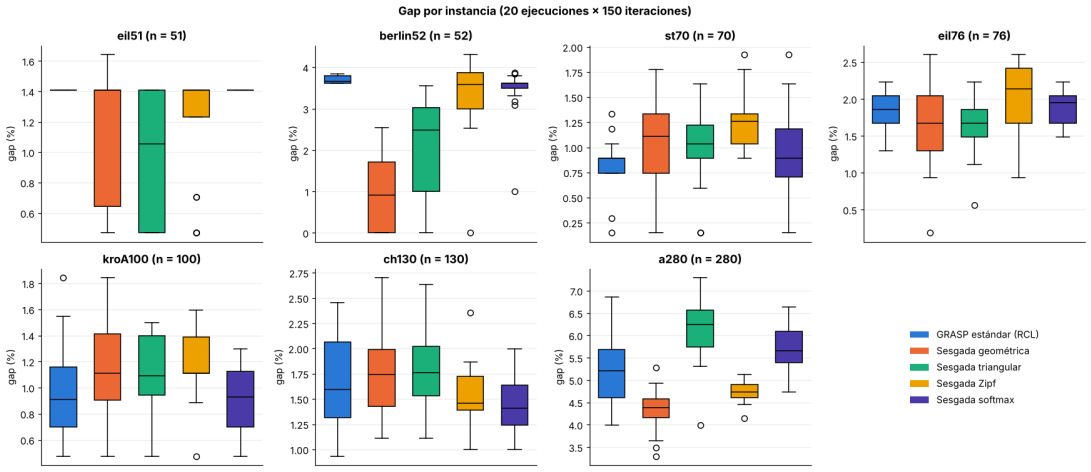


Figure 1: Gap (%) of the 20 runs per instance and rule (boxes: quartiles and median; whiskers and circles: spread and outliers). Colours, left to right: RCL (blue), geometric (orange), triangular (green), Zipf (yellow) and softmax (purple). The title and legend are in Spanish (*GRASP estándar* = RCL, *Sesgada* = biased).

flat blue box in Figure 1), and on `berlin52` the best of 20 runs is still 3.62 %. A rank-based rule fixes the probability of each position whatever the values are.

The other rules. The triangular rule is mixed: better on `eil51` and `berlin52`, worse on `a280`, possibly because $f = 0.002$, tuned on 100–150 cities, admits 78 positions at the first step of `a280` against 10 on `kroB100`. Zipf is never significantly better than the RCL and is worse on `st70`. Softmax never differs significantly from the RCL, and its `eil51` box is also flat. Softmax and the RCL use the same normalised value z_k, and with $T = 0.02$ a candidate at $z_k = -0.05$, the edge of the RCL with $\alpha = 0.05$, weighs $e^{-2.5} \approx 0.08$ relative to the best one: softmax is a smooth version of the same RCL.

Time. A run takes on average 0.14–10.18 s, growing with n. The geometric and triangular rules cost the same as the RCL or less (about 0.8–0.95 times its time). Zipf and softmax, which weigh every feasible pair at every step, need about 1.3–1.75 times the RCL time. Times are means of 20 runs on one machine, comparable only with each other.

Statistical caution. The Friedman test over the seven instances gives $\chi^2 = 1.87$, $p = 0.76$: no global difference between the rules is detected. The evidence for the geometric rule is moderate (two significant wins, none against), and the parameters were tuned on three instances with few runs.

6 Conclusions

A GRASP built on the savings list more than halves the gap of the greedy savings tour followed by 2-opt (5.07 to 2.22 % with the classic RCL). Replacing the uniform RCL by a geometric distribution over ranks lowers the mean gap to 1.70 % at no extra cost, probably because its diversity does not depend on how the saving values are distributed; Zipf, softmax and the triangular rule do not consistently improve on the RCL. The evidence is moderate: two significant wins and a non-significant Friedman test. The next steps are larger instances and more runs, a comparison at equal CPU time, and a stronger local search, which may shrink the differences between rules.

Code availability. All computations are reproducible from the notebook `GRASP_TSP_savings_biased_EN.ipynb`, delivered separately.

References

[1] G. Clarke and J. W. Wright. Scheduling of vehicles from a central depot to a number of delivery points. *Oper. Res.*, 12(4):568–581, 1964.

[2] T. A. Feo and M. G. C. Resende. Greedy randomized adaptive search procedures. *J. Global Optim.*, 6:109–133, 1995.

[3] A. A. Juan, J. Faulín, J. Jorba, D. Riera, D. Masip and B. Barrios. On the use of Monte Carlo simulation, cache and splitting techniques to improve the Clarke and Wright savings heuristics. *J. Oper. Res. Soc.*, 62(6):1085–1097, 2011.

[4] G. Reinelt. TSPLIB — a traveling salesman problem library. *ORSA J. Comput.*, 3(4):376–384, 1991.